picoDAW: a web-based sequencer/synthesizer/sampler

Tim Fan (SID: 550790019)
hourlilies
https://github.com/hourlilies/elec5305-project-550790019

I. Project Overview

This project aims to develop a music sequencer with a synthesizer and sampler as instruments—together forming a very basic digital audio workstation (DAW), hence the name picoDAW—for any device capable of running a modern web browser supporting WebAudio, using the emscripten compiler toolchain that turns C++ DSP code into WebAssembly. The goal is to make music production more accessible, because pretty much any computer, whether that be a smartphone, laptop, or even a single-board computer, can run a modern web browser that supports WebAudio.

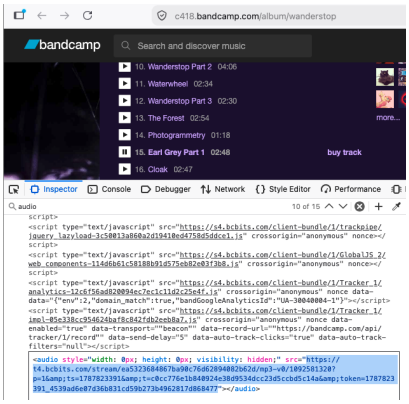
This is important because anyone should be able to play with and make music without needing to install and learn a comprehensive and fully-featured desktop-based DAW application, designed for professional musicians and not the general public.

Though the advent of generative AI in music creation has lowered the barrier of entry tremendously, empowering authors to make and tune music to their taste with just a few text prompts and a bit of luck, it has come at the cost of abstracting away the underlying signal processing and audio intuitions that makes it all work. This project aims to expose all of that, the wires, knobs, and switches of music making, with the hopes of allowing the prospective musician to see and poke at the machine that makes modern music tick, whose human invention and decades of production is what allows powerful GenAI tools like Suno or Udio to even exist.

II. Background and Motivation

Despite its relatively recent history, with the first publication of a working draft in only 2011[1], the WebAudio API has seen extensive development over the course of a decade, eventually receiving an endorsement from the W3C in 2021 as a Recommendation[2]. Notably, its primary `AudioContext` interface has been fully supported by most major web browsers since at least 2015[3], with the only exception being Safari which implemented support in 2021[4].

Of course, audio has existed for a much longer time than that on the internet. As Boris Smus describes in their brief history of audio on the web [1], initially that was through the Flash external plugin for browsers, and then later through the HTML5 `<audio>` element which was instead natively supported by the web browser itself. In fact, it's this `<audio>` element that allows music streaming websites like Bandcamp to preview an artist's music through the web browser (Fig. 1).




```
Input ... from '1092581320.mp3' Duration:
  00:02:48.00, ..., bitrate: 236 kb/s
```

Fig. 1. Bandcamp allows artists to offer their listeners a compressed and lower bitrate MP3 preview of their tracks, played in the browser through the HTML5 `<audio>` element. The text above is the result of `ffprobe 1092581320.mp3`. The author has purchased this album on Bandcamp, buying access to higher fidelity exports of the same songs available as free lower fidelity previews

But one thing Flash's plugin did for web audio that HTML5's `<audio>` element doesn't do, is allow for interactive audio. For example, a synthesizer/sampler/sequencer like the one this project will aim to build, would require a predictable amount of latency between a person pressing a key on their keyboard and the corresponding sound being played through their speakers, something that the `<audio>` element just wasn't designed for.

That is the kind of problem the WebAudio API attempts to solve, and why it has been possible for synthesizers and other real time instruments to be built for the web. For example Steven Goldberg's emulation of the Juno-106 synthesizer[5] built for the web in 2015, or Takamitsu Endo's collection of synthesizers[6] built for the web starting in the 2020's which use the aforementioned `AudioContext` interface as the backbone of its audio framework (Fig. 2); and many

[1] https://www.w3.org/TR/2011/WD-webaudio-20111215/
[2] https://www.w3.org/TR/webaudio-1.0/
[3] https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API#browser_compatibility
[4] https://developer.apple.com/documentation/safari-release-notes/safari-14_1-release-notes#WebKit-Framework
[5] https://github.com/stevengoldberg/juno106
[6] https://github.com/ryukau/UhhyouWebSynthesizers

more examples, like all the web synthesizers available at playtronica[7].

```
// Copyright Takamitsu Endo (ryukau@gmail.com)
// SPDX-License-Identifier: Apache-2.0

export class Audio {
<<<<<<< HEAD
  cachedBuffer = null;
  constructor() {
    // AudioContext is the backbone of WebAudio
    this.audioContext = new AudioContext();

    // Custom class that represents .wav data
    this.wave = new Wave();
  }
  play() {
    if (!this.cachedBuffer) {
      // Create and write .wav data to buffer
      let buffer = this.audioContext.createBuffer(
        this.audioContext.sampleRate);
      for (let i = 0; i < this.wave.channels; ++i) {
        buffer.copyToChannel(new Float32Array(
          this.wave.data[i]), i, 0);
      }
      this.cachedBuffer = buffer;
    }

    // Set AudioContext's buffer to our one
    this.source =
      this.audioContext.createBufferSource();
    this.source.buffer = cachedBuffer;

    // Play the .wav data through AudioContext
    this.source.start();
  }
=======
  cachedBuffer = null;
  constructor() {
    // AudioContext is the backbone of WebAudio
    this.audioContext = new AudioContext();

    // Custom class that represents .wav data
    this.wave = new Wave();
  }
  play() {
    if (!this.cachedBuffer) {
      // Create and write .wav data to buffer
      let buffer = this.audioContext.createBuffer(
        this.audioContext.sampleRate);
      for (let i = 0; i < this.wave.channels; ++i) {
        buffer.copyToChannel(new Float32Array(
          this.wave.data[i]), i, 0);
      }
      this.cachedBuffer = buffer;
    }

    // Set AudioContext's buffer to our one
    this.source =
      this.audioContext.createBufferSource();
    this.source.buffer = cachedBuffer;

    // Play the .wav data through AudioContext
    this.source.start();
  }
>>>>>>> refs/remotes/origin/master
}
```

Fig. 2. Heavily redacted version of the custom Audio class used by ryukau[6] for their web synthesizer plugins, built on the WebAudio API and located at `common/wave.js`

There are also examples of projects for the web that combine a synthesizer with an integrated sequencer, in the same vein as what this project aims to do. One notable example is certainly André Michelle's opendaw[8], which combines a traditional DAW-style sample and MIDI piano-roll sequencer with a collection of instruments and sounds. But while it demonstrates that a comprehensive web-based music production environment is possible, its interface still closely resembles that of conventional desktop DAWs, which I argue does not necessarily make it any more accessible or approachable than its native counterparts.

Another example are web-based grooveboxes TODO TODO GROOVEBOX (pads, things like that)

Thus, this project argues the need for a web-based DAW-like application, that at the same time does not resemble the traditional fully-featured desktop-based DAW it shares the same name with.

DS-10

III. Proposed Methodology

methodology.tex

IV. Expected Outcomes

outcomes.tex

V. Timeline

timeline.tex

References

[1] B. Smus, *Web audio API: advanced sound for games and interactive apps*. O'Reilly Media, Inc., 2013.

[7]https://synth.playtronica.com/

[8]https://opendaw.studio